

The Personalized Edge-Based Driver Drowsiness Detection on Raspberry Pi 4 Using Dynamic EAR Calibration

Nguyen Ba Chinh^{1,a)}, Pham Van Hoang Anh Tu^{1,b)}, Le Van Sang^{1,c)}, Dang Van Hieu^{1,d)}

¹FPT University, Hoa Lac High-Tech Park, Km29 Thang Long Boulevard, Hoa Lac District, Hanoi, Vietnam

- a) Email: chinhhlhiep@gmail.com
- b) Email: tubepvhat1604@gmail.com
- c) Email: lesang08092005@gmail.com
- d) Corresponding author: hieudv2@fe.edu.vn

Abstract. A driver fatigue and drowsiness monitoring and warning system for long-haul driving using edge AI requires high accuracy. However, in practice, system performance is affected by multiple factors such as high hardware costs, unstable Internet connectivity, and varying lighting conditions. This study presents an edge AI system implemented on a Raspberry Pi 4 for fatigue warning and drowsiness detection. The system utilizes MediaPipe Face Mesh and computes the Eye Aspect Ratio (EAR) to recognize facial features and eye states. An adaptive EAR threshold calibration technique is applied to personalize detection and reduce errors associated with fixed thresholds. In addition, a real-time decision-making mechanism integrating a forward-fill algorithm is employed to ensure responsiveness even when the face is partially occluded or under poor lighting conditions. Experimental results on a dataset of Vietnamese drivers demonstrate that the system achieves stable real-time performance at approximately 18.9 FPS, with an overall detection accuracy of 86.95%. The proposed approach provides a cost-effective and reliable solution for in-vehicle driver safety systems, particularly suitable for embedded edge device applications.

I. INTRODUCTION

The U.S. National Highway Traffic Safety Administration claims that drowsy driving results in 100,000 car accidents and over 1,500 fatalities annually. Drowsy driving is believed to cause approximately 1,550 deaths, 71,000 injuries, and financial losses totaling \$12.5 billion. Drowsy driving was a contributing factor to 697 deaths in 2019¹. The number of deaths from road traffic accidents has increased sharply over the past 10 years, with over 18,000 traffic accidents and 8,125 deaths in 2018 according to the General Statistics Office of Vietnam. Traffic accidents account for 11% of all deaths in Vietnam and are predicted to become one of the top three causes of death by 2022². Among many contributing factors, driver fatigue and drowsiness are widely recognized as major causes of traffic accidents due to their significant impact on alertness and reaction time³. Therefore, early detection of driver drowsiness plays a crucial role in improving road safety. Currently, methodologies for detecting driver drowsiness can be classified into three main categories: vehicle-based methods, physiological signal-based methods⁴, and image-based methods. Although physiological approaches like EEG and ECG provide excellent precision, they necessitate invasive sensors that compromise driver comfort and practicality⁵. Vision-based systems that assess face traits, such as eye closure and yawning, provide a non-invasive alternative and have gained from recent advancements in deep learning. Vision-based systems have shown promise, but they still have to deal with a number of problems in the real world. These include partial facial blockage, changing lighting conditions, and integrated devices that can't handle a lot of processing power⁶⁻⁸.

This study presents a real-time driver drowsiness detection and alert system that operates offline on a Raspberry Pi 4 embedded platform to address the aforementioned limitations. The main contributions of this work include: (1) the deployment of an ultra-lightweight MediaPipe deep learning model to optimize processing speed on resource-

constrained edge devices such as the Raspberry Pi 4; and (2) the proposal of an Adaptive Threshold algorithm that enables the system to automatically adapt to eye-related features of individual drivers, overcoming the limitations of conventional fixed thresholds and improving accuracy even under occlusions (e.g., eyeglasses), low-light conditions, and inter-subject variability in ocular characteristics.

II. RELATED WORK

The computer vision-based driver fatigue and drowsiness detection systems, which leverage image processing to generate alerts and help reduce traffic accidents, have attracted significant attention from researchers. Existing studies generally follow two main directions: (1) hardware platform optimization and (2) algorithmic improvements for detection.

A. Hardware Platform and Performance

Bringing image processing algorithms to embedded devices has always been a challenge in balancing accuracy and speed. In early studies, Toan et al.⁹ used a Raspberry Pi 3 Model B in conjunction with a Haar Cascade classifier. However, due to the limitations of the Pi 3's CPU clock speed and RAM capacity, this system only achieved low frame rates and was prone to losing face tracking capabilities in changing lighting conditions. Recently, Vadlamudi¹⁰ in his master's thesis conducted an in-depth performance evaluation of the Raspberry Pi 4 Model B. The results showed that with a quad-core Cortex-A72 processor and upgraded RAM capacity, the Pi 4 is capable of running lightweight convolutional neural networks (Lightweight CNNs) and complex image processing tasks without the need for a dedicated hardware accelerator. Based on this, our research selected the Raspberry Pi 4 as the central processor to ensure real-time responsiveness.

B. EAR Detection Algorithm and Threshold

The most common method for determining eye status is to use Eye Aspect Ratio (EAR), a quantity that does not change with distance and head rotation angle⁹. Sharma et al.¹¹ in 2025 proposed a system using the Dlib library to detect 68 landmarks on the face. Although Dlib provides high accuracy, its drawback is that it consumes a lot of computational resources and especially the use of a fixed EAR threshold (Fixed Threshold = 0.23). This static approach does not take into account the diversity of eye shapes between races and individuals. Authors Ming and Yunbing¹² proposed the concept of "Perception-Independent Calibration" to address these issues. The method collects EAR values from drivers to construct a Gaussian distribution for the dataset, enabling the determination and separation of optimal thresholds for each individual. Building upon this foundation, this paper further develops the approach by integrating the MediaPipe framework to simplify the processing pipeline, making the system more lightweight and suitable for resource-constrained hardware.

III. EXPERIMENTAL

The real-time driver fatigue warning and drowsiness detection system on edge devices such as the Raspberry Pi 4 proposed in this study aims to reduce latency and enhance flexibility through dynamic threshold analysis. The proposed framework consists of three main stages: (1) hardware configuration; (2) geometric feature extraction of eye states; and (3) a dynamic adaptive thresholding method.

A. Hardware Deployment and System Architecture

This study picked the Raspberry Pi 4 Model B as the central processor based on experimental evaluations of computing performance on Vadlamudi's embedded devices. This decision addresses the hardware resource constraints experienced in prior implementations on the Raspberry Pi 3 by Toan et al.¹¹. To assess algorithmic performance under ideal conditions and precisely replicate the offline operating environment within the vehicle, the study team established a controlled testing environment. Specifically, a personal computer using the integrated webcam module acted as a video capture node, connected directly to the Raspberry Pi via the IEEE 802.3 Ethernet protocol.

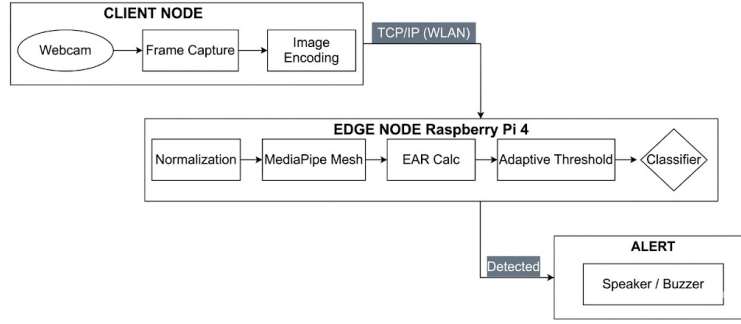


FIGURE 1. System architecture diagram.

B. Face Grid and Landmark Detection

Instead of using traditional face detection methods such as Haar Cascade or HOG combined with SVM as in Sharma's study¹¹, this study applies the MediaPipe Face Mesh framework. Older methods often suffer from high latency and poor stability when the face is tilted or lighting conditions change. In contrast, MediaPipe uses a neural network architecture optimized for mobile devices, allowing real-time extraction of 468 3D landmarks on the face at a stable frame rate. From this point map, the system extracts the coordinates of 6 feature points around each eye ($p1 \rightarrow p2$) to calculate the eye-open state.

To address the problem of head-nodding during driver detection, the study proposes a "Forward-Fill" mechanism to maintain the continuity of the monitoring state. The proposed algorithm utilizes the EAR value from the most recent valid frame to compensate for missing data, instead of reverting the system to the default "Awake" state during signal loss—a commonly used approach that may introduce errors. Specifically, if the EAR indicates that the eyes were in a closed state (low EAR) prior to signal loss, the warning is maintained consistently throughout the tracking interruption period. This helps the system minimize missed drowsiness events (false negatives), particularly during critical moments such as driver head-nodding.

C. Eye Aspect Ratio (EAR) Analysis

To quantify the state of the eye from 2D landmarks, we used the Eye Aspect Ratio (EAR) index. According to the biomedical research review of Kolus et al.¹³, EAR is an effective measure because it does not change with respect to camera-to-face distance and head rotation angle. This index is determined by formula (1):

$$EAR = \frac{||P2 - P6|| + ||P3 - P5||}{2||P1 - P4||} \quad (1)$$

In this case, $p1 \rightarrow p2$ are the coordinates of the landmarks around the eye area. EAR will rapidly decrease to 0 when the eye is closed.

D. Personalized Adaptive Threshold Calibration

This is the main technical contribution of the study to address the drawback of using fixed thresholds in related works, such as the 0.23 threshold used by Sharma. Fixed thresholds often lead to high false-positive rates for drivers with congenitally small eye structures. Therefore, we apply the Non-Cognitive Calibration algorithm proposed by Ming and Yunbing¹². The system will automatically collect EAR data series in the initial phase (approximately the first 15.0 seconds). This data is then modeled into a two-peak distribution, corresponding to the open and closed eye states. The clustering algorithm will automatically determine the optimal cutoff threshold (Th_{adapt}) between these two states, ensuring the highest accuracy for each individual.

To eliminate camera noise or poor performance during initial startup, we designed a system to automatically remove frames with an EAR value less than or equal to 0.05. We will take the average EAR value when the driver's eyes are open (referred to as $\mu_{baseline}$). Multiply this value when the eyes are normally open by 0.8. This method will calculate Th_{adapt} from the following formula: $Th_{adapt} = \mu_{baseline} \times 0.8$.

E. Temporal Decision Logic

According to Sharma ¹¹, the time of a normal blinking action is less than 400ms. This means that if $EAR < Th_{adapt}$ for less than 400ms, it is just a normal blinking action. We designed the algorithm so that when $EAR < Th_{adapt}$ for 1.2 seconds, the system will trigger an alarm.

We measured an average of 18.9 FPS on our Raspberry Pi 4 hardware when running this algorithm. It can be understood that this 1.2-second rule is equivalent to 23 consecutive frames.

We considered real-world scenarios such as: when the driver checks the mirrors, or turns their head to a different angle to react to a situation, or due to vehicle shaking, or even the driver falling asleep. In such cases, the system would lose track of the face and would lack the data to make a decision. Therefore, we proposed Forward-Fill: When the system fails to recognize a face, it will automatically copy the state of the most recent frame to fill in the gaps, but only for 5 consecutive frames.

F. Dataset and Experimental Setup

The control dataset originally recruited 35 Vietnamese individuals (aged 20-30). The study population consists of Southeast Asian subjects, characterized by biometric features such as smaller eye sockets and pronounced eye-corner wrinkles, which help reduce common misdetections when applying standards developed based on Western populations.

The collected dataset includes 62 samples under controlled conditions to evaluate and analyze the accuracy of facial features. Additionally, one of the main challenges encountered involves low-light conditions combined with occlusion. To address this, we applied a computer vision-based brightness reduction technique to transform 15 daytime cases with eyeglasses into simulated low-light data.

In total, the dataset comprises 124,988 frames and 77 video sequences. It is divided into four experimental scenarios: 26 daytime cases without glasses (46,442 frames); 15 daytime cases with glasses (21,837 frames); 19 low-light cases without glasses (32,123 frames); and 17 low-light cases with glasses (24,586 frames), along with 2 real-world recorded cases.

IV. RESULTS AND DISCUSSION

Figure 2 visually illustrates the frame rate (FPS) of the system on the Raspberry Pi 4. The results show an average FPS of 18.9. The reason for the initial FPS drop is that during startup, the system has to load related libraries such as MediaPipe, and the camera needs a few seconds to automatically adjust the exposure. After stabilization, the FPS will increase, sometimes reaching 21 FPS.

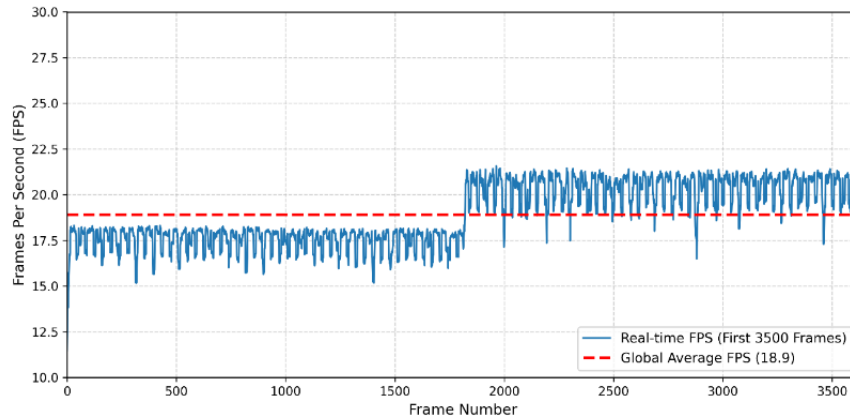


FIGURE 2. Frame rate chart of the system on Raspberry Pi 4

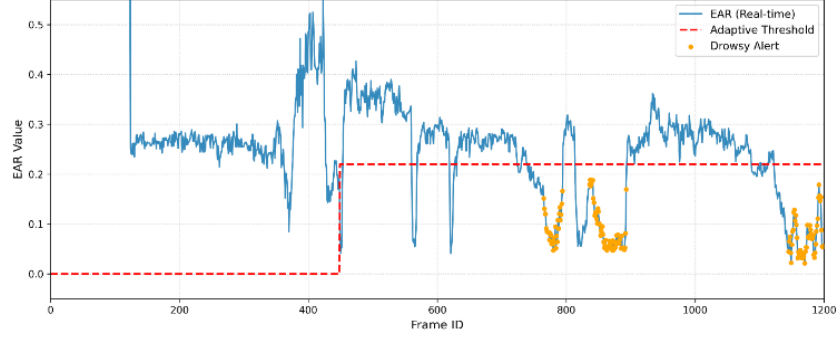


FIGURE 3. Variation of the EAR index among the subjects

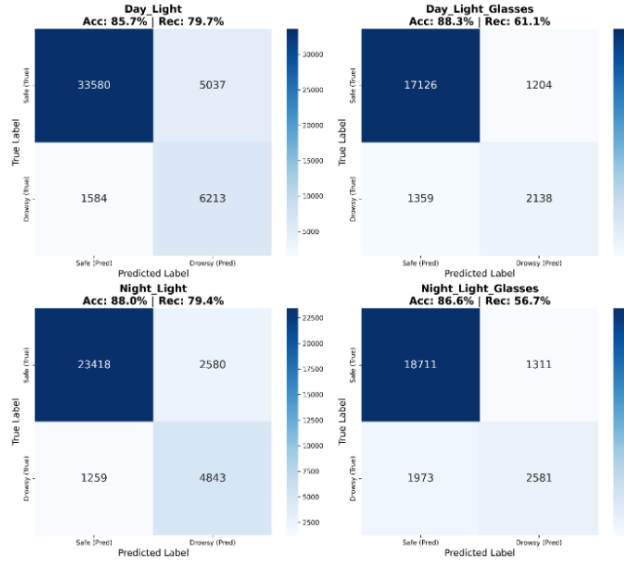


FIGURE 4. Error matrix of the system under four environmental conditions proposed in the study

To help readers better understand how the threshold is calculated for each person, we represent the frame rate over time in Figure 3. The adaptive threshold will be 0 for the first 15 seconds. After the system calculates the threshold for each person, after 15 seconds, the threshold will be updated according to the logic mentioned above: $Th_{adapt} = \mu_{baseline} \times 0.8$. The yellow dots indicate when the system detects drowsiness, meaning that when the driver's EAR index is less than Th_{adapt} for at least 1.2 seconds, the system will trigger an alert.

TABLE 1. comparing performance with related works and the system proposed in this paper ⁹⁻¹².

Study (Ref)	Hardware	Column Header Goes	Acc (%)	FPS	Critical limitation
Thanh ⁹	RPi 3	SVM	>86%	Low	Unstable in lowlight; old hardware
Vadlamudi ¹⁰	RPi 4	CNN+LSTM	~93%	0.86	High Latency; not real-time
Ming ¹²	Xavier	Adaptive	>95%	34	High cost: Requires expensive AI hardware
Sharma ¹¹	RPi 4	Fixed (0.23)	85%	30	Low Accuracy: False Positives or Asian eyes
Ours	RPi 4	Adaptive	86.95%	18.9	Best Trade-of: Real-time on low-cost device

We tested the system under four different conditions. The performance results are shown in Figure 4 under the following conditions: Daylight, Daylight Glasses, Nightlight, and Nightlight Glasses. Under normal conditions

("Day_Light"), the system achieved a recall rate of 79.68, and 79.37 under low light ("Night_Light"). This shows that the system operates effectively when visibility is not obstructed by glasses. When wearing glasses, the recall rate drops sharply, with the lowest recall rate of 56.68% under low-light ("Night_Light Glasses"). We observed that when the driver wears glasses, the light is reflected, obscuring the eyes and preventing the system from recognizing the driver. This highlights the limitations of the RGB sensor. The system only uses EAR (Ear-based detection), so it is less effective when the eyes are obscured. Although lightweight and stable, the system needs additional IR and MAR cameras for improvement. The system achieved 18.9 FPS, showing a good balance between speed, accuracy, and cost.

Compared to Sharma ¹¹, which used a fixed threshold of 0.23 resulting in lower accuracy for people with small eye features such as Asians, our method using an adaptive threshold increased accuracy from 85.5% to 86.95%. It also showed that it is more feasible than expensive hardware solutions like Ming's Nvidia Xavier ¹².

CONCLUSION

This paper proposes a real-time driver drowsiness detection system using inexpensive hardware such as the Raspberry Pi 4. The system combines EAR (Exposure Proximity) and individualized adaptive thresholding to personalize and increase accuracy for each individual.

The results show the system operates effectively, achieving a speed of 18.9 FPS and an accuracy of 86.95, meeting real-world requirements in outdoor environments.

The system's weakness lies in the fact that drivers wearing glasses experience light reflections that obscure their vision. The system performs poorly when the driver's eyes are not visible. Accuracy is almost zero if the driver wears dark glasses.

In the future, to achieve better results, the system should incorporate an infrared camera and MAR to increase accuracy and practical application.

REFERENCES

1. J. Singh, R. Kanojia, R. Singh, R. Bansal and S. Bansal, arXiv preprint arXiv:2303.06310 (2023).
2. H. M. Vu, T. H. Tran, A. K. Dang, T. N. Hoang, C. T. Nguyen, H. L. T. Nguyen, C. A. Latkin, C. S. Ho and R. C. Ho, Scientific reports **13** (1), 11496 (2023).
3. A. Sahayadhas, K. Sundaraj and M. Murugappan, Sensors **12** (12), 16937-16953 (2012).
4. G. Li and W.-Y. Chung, Sensors **22** (3), 1100 (2022).
5. J. S. Bajaj, N. Kumar, R. K. Kaushal, H. Gururaj, F. Flammini and R. Natarajan, Sensors **23** (3), 1292 (2023).
6. B. Bakker, B. Zabłocki, A. Baker, V. Riethmeister, B. Marx, G. Iyer, A. Anund and C. Ahlström, IEEE Transactions on Intelligent Transportation Systems **23** (5), 4791-4800 (2021).
7. R. Florez, F. Palomino-Quispe, A. B. Alvarez, R. J. Coaquira-Castillo and J. C. Herrera-Levano, Sensors **24** (19), 6261 (2024).
8. F. Safarov, F. Akhmedov, A. B. Abdusalomov, R. Nasimov and Y. I. Cho, Sensors **23** (14), 6459 (2023).
9. T. D. Thanh and V. T. Linh, Tạp chí Khoa học Giao thông vận tải **70** (3), 184-192 (2019).
10. S. Vadlamudi, Master's Thesis, 2020.
11. A. Sharma, G. Verma, M. Mittal, M. Garg and M. Rani, International Journal of Scientific Development and Research (IJS DR) **10** (3), 472-477 (2025).
12. C. Ming and Y. Yunbing, IEEE Access **10**, 125469-125476 (2022).
13. A. Kolus, IEEE Access **12**, 97969-97993 (2024).